Orientação a Objetos: material explicativo

Guia ampliado alinhado à Matriz de Referência

Data: 09/09/2026

Tema: Orientação a Objetos

Sumário

1 Apresentação e modo de uso 1
1.1 Referencial teórico 1
1.2 Como estudar cada tópico 1
2 Leitura da matriz e objetivos de aprendizagem 2
2.1 Glossário essencial 3
3 Unidade 1 — Fundamentos e fronteiras 4
3.1 Tópico 1 — Classes, objetos e operações 4
3.1.1 Classe, objeto, identidade, estado e comportamento 4
3.1.2 Exemplo: uma conta que preserva sua invariante 4
3.1.3 Sobrecarga, igualdade e operações estáticos 5
3.1.4 Esquema de abstração e despacho 5
3.1.5 Decisões de projeto 5
3.1.6 Verificação rápida 6
3.2 Tópico 2 — Classes abstratas e interfaces 6
3.2.1 Duas formas de abstração 6
3.2.2 Exemplo: depender do contrato 7
3.2.3 Critério de escolha 7
4 Unidade 2 — Abstração, herança e variação 8
4.1 Tópico 3 — Encapsulamento 8
4.1.1 Encapsular não é apenas esconder campos 8
4.1.2 Exemplo: não vazar a representação 8
4.1.3 Checklist de encapsulamento 8
4.2 Tópico 4 — Herança simples e múltipla 9
4.2.1 Três sentidos de herança 9
4.2.2 Exemplo Java: especialização coesa 9
4.2.3 Herança múltipla por interfaces 10
4.2.4 Quando preferir composição 10
4.3 Tópico 5 — Polimorfismo 10
4.3.1 Tipo estático, classe dinâmica e despacho 10
4.3.2 Subtipagem comportamental 11
5 Unidade 3 — Modelagem estrutural e relações 11
5.1 Tópico 6 — Diagramas de classes UML e multiplicidade 11
5.1.1 O que um diagrama de classes modela 11
5.1.2 Como ler multiplicidade 11
5.1.3 Procedimento de modelagem 12
5.1.4 Classe de associação 12
5.1.5 Limitações da UML 12
5.2 Tópico 7 — Associação, agregação e composição 12
5.2.1 Associação 12
5.2.2 Agregação compartilhada 13

5.2.3 Composição 13
6 Unidade 4 — Coordenação, padrões e integração 13
6.1 Tópico 8 — Padrões GoF: Observer, Mediator e Singleton 13
6.1.1 Como ler um padrão 13
6.1.2 Observer — notificação extensível 14
6.1.3 Mediator — coordenação centralizada 15
6.1.4 Singleton — unicidade em escopo real 16
6.1.5 Comparação dos três padrões 17
6.2 Como escolher um padrão sem decorar nomes 17
7 Exemplos integradores 17
7.1 Integração 1 — Carrinho, composição e Strategy 17
7.2 Integração 2 — Pedido, UML e invariantes 18
7.3 Integração 3 — Checkout com Mediator e Observer 19
7.4 Integração 4 — Biblioteca e classe de associação 19
8 Roteiro de atividades e avaliação 19
8.1 Atividade 1 — Diagnóstico de conceitos 19
8.2 Atividade 2 — Refatoração por composição 20
8.3 Atividade 3 — Validação de diagrama UML 20
8.4 Rubrica de avaliação 20
9 Síntese final e checklist 21
10 Referências 21
10.1 Nota sobre o alinhamento 22

1 Apresentação e modo de uso

Este material foi elaborado a partir da planilha **Matriz de Referência** fornecida pelo solicitante. A matriz define o tema **Orientação a Objetos**, organiza oito tópicos em quatro unidades de aprendizagem e relaciona os tópicos a quatro objetivos de aprendizagem. O texto a seguir é deliberadamente mais amplo do que um conjunto de slides: além das definições, explicita decisões de projeto, exemplos em Java, modelos UML, esquemas, gráficos, armadilhas e atividades de verificação.

O foco não é memorizar nomes de recursos da linguagem. O foco é aprender a **modelar responsabilidades**, expor contratos estáveis, preservar invariantes, escolher relações com semântica adequada e avaliar os custos de cada decisão. Sempre que a planilha propõe um conceito, o material acrescenta a pergunta de projeto que permite aplicá-lo em uma situação concreta.

> **Tese orientadora**
> Um bom projeto orientado a objetos expõe contratos comportamentais estáveis, preserva invariantes, atribui responsabilidades coesas e escolhe a menor semântica suficiente: associação antes de composição, composição antes de herança quando a subtipagem não for necessária e padrão GoF somente quando o problema justificar a estrutura adicional.

1.1 Referencial teórico

A fundamentação combina quatro camadas. A primeira é a teoria de tipos, abstração e polimorfismo, especialmente a distinção entre tipo, implementação e comportamento discutida por Cardelli e Wegner [1]. A segunda é a modularidade por ocultação de informação, associada a Parnas [2] e Snyder [3]. A terceira é a subtipagem comportamental, para a qual Liskov e Wing oferecem uma disciplina de substituibilidade [4]. A quarta é a modelagem visual e a reutilização de soluções recorrentes, apoiadas respectivamente pela especificação UML do Object Management Group [5] e pelo catálogo GoF [6].

As afirmações específicas sobre Java são tratadas como afirmações normativas e referenciadas à **Java Language Specification** e à documentação oficial da plataforma [7] [8] [9]. As recomendações de design são apresentadas como critérios e trade-offs. Elas não são leis universais: uma escolha aceitável depende do domínio, do ciclo de vida, da concorrência, dos testes e do custo de manutenção.

1.2 Como estudar cada tópico

Para cada conceito, siga a sequência: (1) defina o termo; (2) escreva a regra ou contrato em linguagem natural; (3) modele a responsabilidade; (4) implemente a menor solução que satisfaça o contrato; (5) teste casos válidos e inválidos; (6) registre o custo da decisão. A sequência evita a prática de desenhar classes ou aplicar padrões apenas porque uma palavra do enunciado parece sugeri-los.

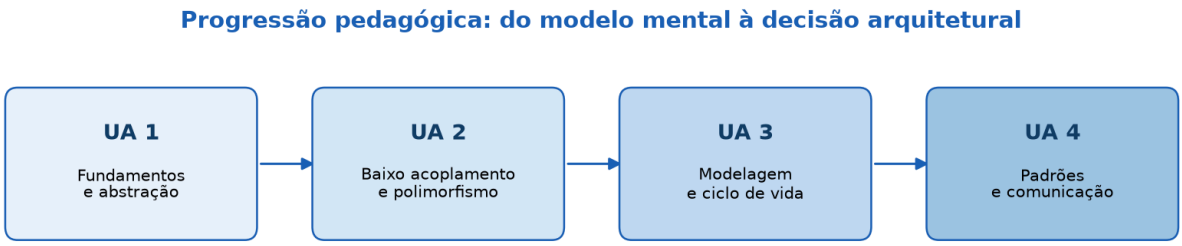


Figura 1: Progressão recomendada: dos fundamentos e fronteiras às decisões de coordenação e padrões.

2 Leitura da matriz e objetivos de aprendizagem

A planilha distribui os tópicos em quatro unidades. A organização abaixo preserva o conteúdo original e torna explícitas as dependências conceituais entre eles.

Unidade	Tópicos	Pergunta condutora	Evidência de aprendizagem
1	1. Classes, objetos e operações; 3. Encapsulamento	O que um objeto sabe, faz e deve proteger?	Implementação de `Conta` ou `Carrinho`, testes de fronteira e diagrama de classes simples.
2	2. Classes abstratas e interfaces; 4. Herança simples e múltipla; 5. Polimorfismo	Como variar implementações sem quebrar o cliente?	Hierarquia `Forma`, contrato de serviço e função polimórfica sem `instanceof` desnecessário.
3	6. UML e multiplicidade; 7. Associação, agregação e composição	Como tornar explícitas as conexões e seus limites?	Diagrama de vendas ou biblioteca, casos mínimos/máximos e implementação encapsulada.
4	8. Padrões GoF: Observer, Mediator e Singleton; integração	Qual problema de acoplamento o padrão resolve e qual custo introduz?	Projeto com evento, mediador, configuração injetada, testes e justificativa de trade-offs.

A matriz associa quatro objetivos de aprendizagem a essas unidades. O **OBJ-1** trata de classes, objetos, atributos, operações, instanciação, classes abstratas e interfaces. O **OBJ-2** trata de encapsulamento, herança, polimorfismo dinâmico, baixo acoplamento e comparação entre composição e herança. O **OBJ-3** trata de UML, multiplicidade, associações e ciclo de vida. O **OBJ-4** trata da classificação e aplicação de padrões GoF, com destaque para Observer, Mediator e Singleton.

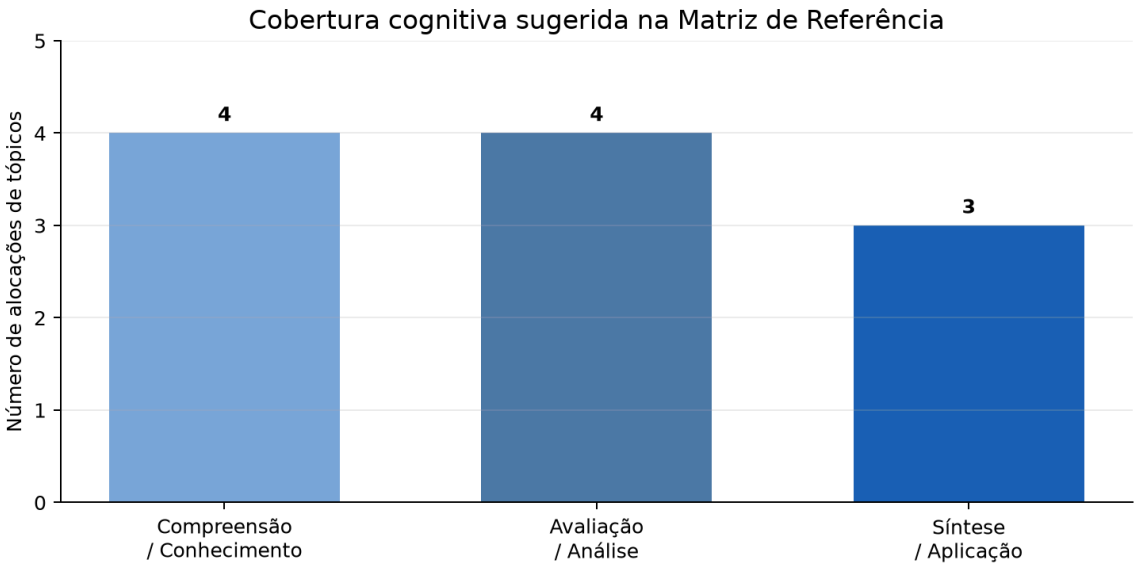


Figura 2: Distribuição sugerida de alocações de tópicos nos níveis cognitivos da planilha. A contagem é uma leitura da própria matriz, não uma medida de dificuldade.

A cobertura proposta pela planilha contém quatro alocações na faixa de compreensão/conhecimento, quatro na faixa de avaliação/análise e três na faixa de síntese/aplicação. Isso sugere uma avaliação que não deve se limitar à definição de termos: o estudante precisa interpretar relações, diagnosticar problemas de acoplamento e propor soluções justificadas.

2.1 Glossário essencial

Termo	Definição operacional	Pergunta de controle
Classe	Tipo que descreve estado, operações, construtores e regras de acesso.	Que invariantes e responsabilidades pertencem a este tipo?
Objeto	Instância concreta com identidade, estado atual e comportamento acessível.	O que diferencia esta instância das demais?
Operação	Operação com parâmetros, retorno, contrato, implementação e possíveis efeitos.	Qual pré-condição e pós-condição a operação garante?
Encapsulamento	Fronteira que reúne estado e comportamento e controla a interação com clientes.	Que transições inválidas a API impede?
Subtipo	Tipo cujas instâncias podem substituir as do tipo base sem quebrar expectativas.	O cliente continua correto com a implementação concreta?
Multiplicidade	Intervalo mínimo–máximo de valores associados a uma instância.	Dado um objeto, quantos objetos do outro lado são permitidos?
Observer	Notificação de muitos receptores inscritos após mudança ou evento.	Como inscrever, cancelar e tratar falhas?
Mediator	Coordenação centralizada entre componentes que poderiam ter referências cruzadas.	A coordenação está coesa ou virou um objeto central excessivo?

Singleton	Controle de uma única instância em um escopo definido.	A unicidade é requisito real ou apenas conveniência de acesso?

3 Unidade 1 — Fundamentos e fronteiras

3.1 Tópico 1 — Classes, objetos e operações

3.1.1 Classe, objeto, identidade, estado e comportamento

Uma classe é um tipo e uma unidade de descrição. Ela pode declarar atributos, operações, construtores, classes internas e regras de acesso. Um objeto é uma ocorrência concreta desse tipo. A classe estabelece o vocabulário e as restrições; o objeto mantém valores específicos em determinado momento.

É útil separar três ideias. **Identidade** é aquilo que permite distinguir um objeto de outro mesmo quando os valores são iguais. **Estado** é o conjunto de valores relevantes naquele momento. **Comportamento** é o conjunto de operações oferecidas pelo objeto. A metáfora de "objeto do mundo real" ajuda no início, mas é limitada: as classes de um software dependem do problema, das responsabilidades e dos contratos que precisam ser mantidos.

Um operação tem nome, parâmetros, tipo de retorno, visibilidade, contrato e implementação. O corpo não esgota o significado do operação: o cliente também depende de pré-condições, pós-condições, invariantes e efeitos observáveis. Em Java, o construtor participa da criação e inicialização; ele não é um operação chamado depois da criação. A assinatura usada para sobrecarga não inclui o tipo de retorno [7] [9].

3.1.2 Exemplo: uma conta que preserva sua invariante

A regra `saldoCentavos >= 0` é uma invariante da classe. A implementação não expõe um `setSaldo` irrestrito; expõe operações que representam ações do domínio.

```
public final class Conta {
    private final String id;
    private long saldoCentavos;

    public Conta(String id, long saldoInicialCentavos) {
        if (id == null || id.isBlank())
            throw new IllegalArgumentException("id obrigatório");
        if (saldoInicialCentavos < 0)
            throw new IllegalArgumentException("saldo inicial negativo");
        this.id = id;
        this.saldoCentavos = saldoInicialCentavos;
    }

    public String id() { return id; }
    public long saldoCentavos() { return saldoCentavos; }

    public void depositar(long valorCentavos) {
```

```
        exigirValorPositivo(valorCentavos);
        saldoCentavos += valorCentavos;
    }

    public void sacar(long valorCentavos) {
        exigirValorPositivo(valorCentavos);
        if (valorCentavos > saldoCentavos)
            throw new IllegalStateException("saldo insuficiente");
        saldoCentavos -= valorCentavos;
    }

    private static void exigirValorPositivo(long valor) {
        if (valor <= 0)
            throw new IllegalArgumentException("valor inválido");
    }
}
```

O construtor estabelece uma condição inicial válida. `depositar` e `sacar` verificam suas pré-condições e preservam a invariante. A escolha de centavos inteiros torna o exemplo financeiro didático independente de erros de arredondamento de ponto flutuante.

3.1.3 Sobrecarga, igualdade e operações estáticos

Sobrecarga ocorre quando operações têm o mesmo nome e parâmetros diferentes; a seleção é feita em compilação. Duas declarações que diferem apenas pelo retorno não constituem sobrecarga válida. Em Java, == para referências testa identidade. `equals` expressa igualdade lógica quando a classe define esse contrato. Se `equals` for redefinido, o contrato de `hashCode` também precisa ser respeitado para uso em estruturas baseadas em hash [8].

Operações `static` pertencem ao tipo e não participam do despacho polimórfico comum de operações de instância. Uma referência `final` não torna automaticamente imutável o objeto referenciado. Uma coleção privada também pode vazar se um getter devolver a própria coleção modificável.

3.1.4 Esquema de abstração e despacho

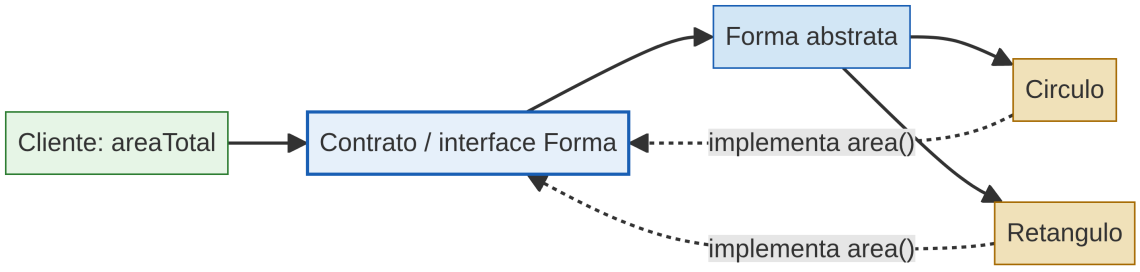


Figura 3: Uma referência do cliente pode apontar para diferentes implementações compatíveis com o contrato `Forma`.

3.1.5 Decisões de projeto

Decisão	Critério recomendado	Risco a discutir

Campo público ou privado	Privado por padrão; expor operações que expressem regras do domínio.	A representação vira contrato e dificulta mudanças.
Instância ou `static`	Usar instância quando o comportamento depende do estado do objeto.	`static` reduz polimorfismo e pode esconder dependências.
Mutabilidade	Permitir transições necessárias e controladas.	Mais estados possíveis, concorrência e testes mais difíceis.
`equals`/`hashCode`	Definir uma noção de igualdade estável e adequada ao domínio.	Chaves mutáveis podem tornar objetos difíceis de localizar em `HashSet`.
Herança ou composição	Herança para subtipagem estável; composição para combinar capacidades.	A superclasse pode acoplar a detalhes protegidos.

3.1.6 Verificação rápida

- Explique por que classe e objeto não são sinônimos.
- Indique pré-condição, pós-condição e invariante no exemplo `Conta`.
- Diga por que um retorno diferente não torna duas declarações uma sobrecarga válida.
- Explique a diferença entre identidade (`==`) e igualdade lógica (`equals`).

3.2 Tópico 2 — Classes abstratas e interfaces

3.2.1 Duas formas de abstração

Uma classe abstrata não pode ser instanciada diretamente. Ela pode combinar estado, construtores, operações concretos e operações abstratos. Não é obrigatório que toda classe abstrata tenha operação abstrato; o modificador também pode comunicar que a classe é uma base deliberada de uma hierarquia.

Uma interface é um tipo de referência voltado a um contrato, capacidade ou papel. Uma classe Java pode implementar várias interfaces. Em Java moderno, interfaces podem declarar operações abstratos, `default`, `static` e `private` conforme as regras da linguagem. Campos declarados em interfaces são constantes `public static final`, não estado mutável por instância [7] [10].

Critério	Classe abstrata	Interface
Instanciação	Não pode ser instanciada diretamente.	Não se instancia uma interface diretamente.
Estado de instância	Pode declarar campos e invariantes compartilhadas.	Não oferece campos de instância; constantes são outra categoria.
Implementação comum	Pode compartilhar estado, validação e algoritmo parcial.	Pode compartilhar operações `default`, mas não uma base de estado.
Herança em Java	Uma superclasse direta.	Uma classe pode implementar várias interfaces.
Uso típico	Família coesa com estado e implementação comum.	Contrato, capacidade ou papel que atravessa hierarquias.

3.2.2 Exemplo: depender do contrato

```
public interface Exportador {
    String exportar(Relatorio relatorio);
}

public final class ExportadorCsv implements Exportador {
    @Override
    public String exportar(Relatorio relatorio) {
        return relatorio.linhas().stream()
            .map(linha -> String.join(";", linha))
            .collect(java.util.stream.Collectors.joining("\n"));
    }
}

public final class ServicoDeRelatorios {
    private final Exportador exportador;

    public ServicoDeRelatorios(Exportador exportador) {
        this.exportador = java.util.Objects.requireNonNull(exportador);
    }

    public String gerar(Relatorio relatorio) {
        return exportador.exportar(relatorio);
    }
}
```

`ServicoDeRelatorios` depende de `Exportador`, não de `ExportadorCsv`. Um teste pode fornecer uma implementação controlada. Uma classe abstrata seria preferível se as implementações precisassem compartilhar estado protegido, validação e uma sequência de passos. A combinação também é possível: uma interface define o contrato e uma classe abstrata fornece uma implementação esqueleto, como `AbstractList<E>` na biblioteca Java [11].

3.2.3 Critério de escolha

Escolha uma interface quando o cliente precisa de um contrato, quando os implementadores não formam uma única família de estado ou quando múltiplos papéis são relevantes. Escolha uma classe abstrata quando invariantes, estado e algoritmo comum formam uma hierarquia coesa. Use composição quando o compartilhamento de código não representar uma relação de subtipagem.

Um compilador verifica assinaturas e visibilidade; não prova pós-condições, invariantes nem substituibilidade comportamental. Adicionar operação abstrato a uma interface pública pode quebrar implementadores. Um operação `default` reduz parte da ruptura de compilação, mas não resolve automaticamente a compatibilidade semântica.

4 Unidade 2 — Abstração, herança e variação

4.1 Tópico 3 — Encapsulamento

4.1.1 Encapsular não é apenas esconder campos

Encapsulamento é a organização de estado e comportamento sob uma fronteira que controla como clientes interagem com a abstração. **Ocultação de informação** é a decisão de esconder decisões de implementação que podem mudar. **Abstração** seleciona o que o cliente precisa conhecer. Os três conceitos se relacionam, mas não são sinônimos [2] [3].

Java oferece `public`, `protected`, acesso de pacote e `private`. Esse controle é principalmente uma restrição verificada pelo compilador; não é uma barreira criptográfica. O objetivo é limitar dependências, preservar invariantes e tornar a evolução do módulo menos arriscada.

Uma classe com campo privado pode continuar mal encapsulada se expuser setters irrestritos, retornar referências mutáveis internas ou usar `protected` apenas como atalho para subclasses. A pergunta correta não é "há getters e setters?". A pergunta é "quais estados e transições o contrato permite?".

4.1.2 Exemplo: não vazar a representação

```
public final class Agenda {
    private final java.util.List<String> contatos =
        new java.util.ArrayList<>();

    public void adicionar(String contato) {
        if (contato == null || contato.isBlank())
            throw new IllegalArgumentException("contato inválido");
        contatos.add(contato);
    }

    public java.util.List<String> contatos() {
        return java.util.List.copyOf(contatos);
    }
}
```

`List.copyOf` impede que o cliente modifique a coleção interna por meio do retorno. Isso não torna os elementos automaticamente imutáveis e não resolve conjuntos os problemas de concorrência; resolve o vazamento da representação daquela coleção.

4.1.3 Checklist de encapsulamento

1. Qual invariante deve ser sempre verdadeira?
2. Quais operações públicas alteram o estado?
3. Cada operação verifica pré-condições e preserva pós-condições?
4. Alguma referência mutável interna é devolvida ou armazenada sem cópia?
5. `protected` é um contrato deliberado de extensão ou apenas conveniência?
6. O cliente depende de comportamento ou de uma representação concreta?
7. A imutabilidade é real ou apenas a referência é `final`?

Erro recorrente

Encapsulamento não equivale a “atributos privados mais getters e setters”. Uma API pode esconder campos e ainda permitir qualquer transição inválida. O critério é a preservação da abstração e das invariantes.

4.2 Tópico 4 — Herança simples e múltipla

4.2.1 Três sentidos de herança

Herança pode significar herança de **tipo**, de **implementação** ou de **estado**. Uma linguagem pode permitir uma dessas formas sem permitir as outras. Além disso, subclasse e subtipo não são sinônimos. Uma subclasse somente deve ser usada como subtipo quando instâncias dela puderem substituir instâncias do tipo base sem quebrar expectativas documentadas [4].

Herança simples usa uma única superclasse direta. Java permite essa forma para classes e permite que uma classe implemente várias interfaces. C++ permite várias classes-base; bases virtuais tratam o compartilhamento de um subobjeto-base no problema do diamante. Python usa uma ordem de resolução de operações, a MRO C3, para organizar certas hierarquias múltiplas [12] [13] [14].

4.2.2 Exemplo Java: especialização coesa

```java
public abstract class Forma {
    private final String cor;

    protected Forma(String cor) {
        this.cor = java.util.Objects.requireNonNull(cor);
    }

    public String cor() { return cor; }
    public abstract double area();
}

public final class Circulo extends Forma {
    private final double raio;

    public Circulo(String cor, double raio) {
        super(cor);
        if (raio <= 0) throw new IllegalArgumentException("raio inválido");
        this.raio = raio;
    }

    @Override
    public double area() { return Math.PI * raio * raio; }
}
```

O construtor da subclasse inicializa a parte da superclasse por `super(...)`. Construtores não são membros herdados. Operações podem ser sobrescritos; campos podem ser ocultados, e esses fenômenos têm semânticas diferentes. A anotação `@Override` ajuda o compilador a verificar a intenção de sobrescrita.

4.2.3 Herança múltipla por interfaces

```
interface Exportavel { String exportar(); }
interface Auditavel { String eventoAuditoria(); }

final class Relatorio implements Exportavel, Auditavel {
    @Override public String exportar() { return "..."; }
    @Override public String eventoAuditoria() { return "relatorio-exportado"; }
}
```

A formulação rigorosa não é "Java tem" ou "Java não tem" herança múltipla sem qualificação. Java tem herança múltipla de tipo por interfaces, mas não permite múltiplas superclasses de estado e implementação como C++. Se duas interfaces fornecerem o mesmo operação `default`, a classe pode precisar resolver o conflito explicitamente.

4.2.4 Quando preferir composição

Prefira composição ou delegação quando a relação de especialização não for estável, quando a reutilização for apenas de código, quando a classe-base expuser detalhes protegidos ou quando as combinações de capacidades puderem crescer de forma independente. Herança é adequada para uma hierarquia pequena, coesa e documentada como contrato de substituição.

4.3 Tópico 5 — Polimorfismo

4.3.1 Tipo estático, classe dinâmica e despacho

Polimorfismo de subtipos permite usar uma referência a uma abstração para operar sobre objetos concretos diferentes. Polimorfismo paramétrico aparece em generics, como `List<T>`. Sobrecarga usa o nome e os parâmetros disponíveis em compilação. Sobrescrita redefine um operação herdado compatível, e a implementação de instância elegível é escolhida dinamicamente. Operações `static` são ocultados, não despachados como instância [7] [9].

```
public static double areaTotal(java.util.List<Forma> formas) {
    return formas.stream()
        .mapPendênciauble(Forma::area)
        .sum();
}
```

`areaTotal` não precisa conhecer `Circulo` ou `Retangulo`. O cliente depende de `Forma.area()`. Essa é a vantagem prática: a variação fica atrás do contrato.

Fenômeno	Seleção	Despacho dinâmico comum?	Exemplo
Sobrecarga	Nome e parâmetros em compilação.	Não.	`imprimir(int)` e `imprimir(String)`

Sobrescrita	Operação herdado com assinatura compatível.	Sim, para instância elegível.	`Forma.area()` em `Circulo`
Ocultação	Membro `static` redeclarado.	Não como instância.	`Base.nome()` e `Derivada.nome()`
Campo	Tipo declarado da referência.	Não como operação virtual.	`base.valor` pode diferir
Genérico	Tipo parametrizado.	Depende da operação e do apagamento.	`List<Forma>`

4.3.2 Subtipagem comportamental

Uma assinatura compatível não basta. Um subtipo que lança exceção para entradas aceitas pelo tipo base, exige uma pré-condição mais forte ou retorna algo fora da pós-condição pode quebrar o cliente. A análise de Liskov e Wing é uma disciplina de revisão: o sistema deve continuar correto quando a implementação concreta variar [4].

Use `instanceof` e casts quando a distinção de tipo fizer parte do problema e não houver abstração comportamental melhor. Polimorfismo não elimina contratos, testes ou custos de depuração. Em Java, generics usam apagamento de tipos em grande parte da execução, e operações ponte podem ser gerados para preservar polimorfismo após a compilação [15] [16].

5 Unidade 3 — Modelagem estrutural e relações

5.1 Tópico 6 — Diagramas de classes UML e multiplicidade

5.1.1 O que um diagrama de classes modela

Um diagrama de classes UML representa uma visão estrutural: classificadores, atributos, operações, generalizações, associações, dependências e restrições. Ele é um modelo de comunicação e análise, não uma especificação completa de algoritmos, sequência de chamadas ou código gerado automaticamente [5] [17].

Uma classe pode ser representada por um retângulo com nome, atributos e operações. Os sinais de visibilidade costumam ser + para público, - para privado, # para protegido e ~ para visibilidade de pacote. Um diagrama de análise pode omitir detalhes de implementação; um diagrama de projeto pode aproximar-se das assinaturas do código.

5.1.2 Como ler multiplicidade

A multiplicidade é um intervalo inclusivo de cardinalidade. A leitura deve partir de uma instância no extremo oposto. Se `Pedido` está ligado a `Cliente` com `1` no lado de `Cliente`, a regra é: cada pedido se relaciona com exatamente um cliente. Não significa que exista um único cliente no sistema.

Notação	Leitura	Observação
`0..1`	zero ou um	Pode haver ausência ou um único valor.
`1` / `1..1`	exatamente um	A omissão do intervalo igual não altera o significado.
`0..*` / `*`	zero ou muitos	A estrela isolada é abreviação de `0..*`; permite coleção vazia.
`1..*`	um ou muitos	Exige pelo menos um valor.
`m..n`	de m a n	Limites inclusivos; `n` precisa ser maior ou igual a `m`.

A cardinalidade não informa automaticamente ordem, duplicidade, ciclo de vida ou temporalidade. `0..*` pode ser representado conceitualmente por `Set`, `List`, `Bag` ou outra coleção. `ordered` e `unique` são propriedades separadas. Em Java, `Set` evita duplicatas segundo seu contrato; `List` é ordenada e normalmente permite duplicatas [18] [19].

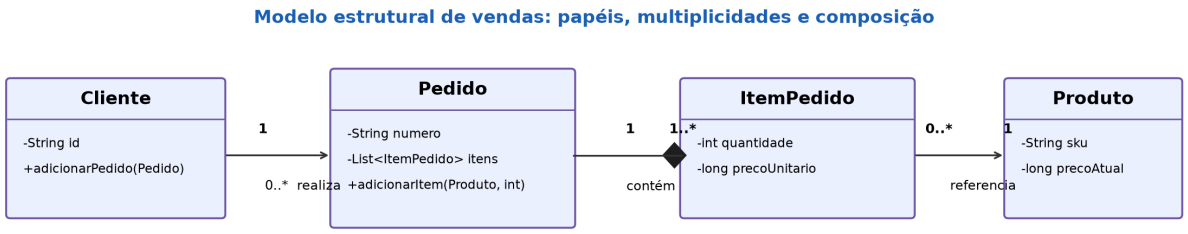


Figura 4: Exemplo de vendas: um cliente realiza zero ou muitos pedidos; cada pedido contém pelo menos um item; cada item referencia um produto.

5.1.3 Procedimento de modelagem

1. Defina a fronteira do modelo: domínio, aplicação ou persistência.
2. Escreva cada associação como uma frase nas duas direções.
3. Pergunte a cardinalidade mínima e máxima em cada extremo.
4. Separe cardinalidade de ordem, unicidade e navegabilidade.
5. Decida se é associação, agregação ou composição e registre a razão.
6. Teste o diagrama com instâncias mínimas, máximas e inválidas.
7. Use OCL, invariantes textuais, restrições de banco ou testes para regras condicionais.

5.1.4 Classe de associação

Quando o relacionamento tem dados próprios, a linha pode virar uma classe. “Aluno cursa disciplina” pode exigir `semestre`, `nota`, `frequencia` e `situacao`; nesse caso, `Matricula` representa o vínculo. A multiplicidade `0..*` dos dois lados não garante que a mesma combinação aluno-disciplina possa ocorrer duas vezes no mesmo semestre. Essa unicidade composta exige uma restrição adicional.

5.1.5 Limitações da UML

Mesmo com multiplicidades preenchidas, podem faltar invariantes, pré-condições, pós-condições, autorização, concorrência, temporalidade e decisões de persistência. A verificação de modelos UML/OCL pode ser difícil quando associações, herança, cardinalidades e invariantes interagem [20]. Um modelo visual torna regras explícitas, mas não substitui o código, os testes ou uma verificação formal quando o risco justificar.

5.2 Tópico 7 — Associação, agregação e composição

5.2.1 Associação

Associação é o relacionamento estrutural geral entre instâncias de classificadores. Ela pode ter papéis, multiplicidades, navegabilidade, atributos de extremidade, qualificadores e uma classe de associação. Em Java, costuma ser representada por referência ou coleção, mas a presença de um campo não determina sozinha a semântica do domínio.

Uma associação pode ser lida como "um cliente realiza pedidos". O cliente pode existir sem um pedido específico, e a API deve manter a consistência dos dois lados se a navegação bidirecional for realmente necessária. Um `Set` ajuda a evitar duplicatas, mas não atualiza automaticamente o outro lado nem garante limites `m..n`.

5.2.2 Agregação compartilhada

Agregação é uma intenção de relação conjunto–parte fraca, tradicionalmente representada por um losango vazio. A UML 2.5.1 não atribui uma semântica operacional universal à agregação compartilhada; a interpretação varia conforme o domínio e o modelador [5]. Por isso, o diamante vazio deve ser usado com cautela e acompanhado de uma regra clara.

`Equipe` e `Jogador` pode ser uma agregação se o modelo quiser comunicar uma relação de grupo sem que a equipe controle a existência do jogador. O jogador pode ser transferido, existir fora de uma equipe ou participar de outros contextos conforme as regras do domínio. Se a informação conjunto–parte não for necessária, uma associação simples é mais precisa.

5.2.3 Composição

Composição é uma agregação forte. A parte pertence a no máximo um composto por vez e o composto assume responsabilidade estrutural pela parte. O diamante preto fica no extremo do conjunto. Em `Pedido–ItemPedido`, a composição é justificável se cada item pertencer a um único pedido e seu ciclo de vida for controlado pelo pedido.

Composição UML não é sinônimo de classe interna, de `new` dentro do construtor ou de destrutor Java. A coleta de lixo não traduz automaticamente todas as decisões de ciclo de vida UML. Use composição somente quando exclusividade e dependência estrutural forem invariantes do domínio.

Relação	Semântica mínima	Pergunta de decisão	Cuidado
Associação	Objetos se relacionam ou colaboram.	Um usa ou conhece o outro?	Não pressupõe posse.
Agregação	Conjunto–parte fraco e contextual.	É útil comunicar partes relativamente independentes?	Não prometer regras que a UML não garante.
Composição	Parte exclusiva e estruturalmente pertencente ao conjunto.	A parte pode ser compartilhada ou sobreviver em outro composto?	Não converter mecanicamente em destruição Java.

Regra editorial

Use a menor semântica suficiente. Uma associação explícita é melhor que uma composição não justificada; muitos diamantes diminuem a precisão quando não há invariantes correspondentes.

6 Unidade 4 — Coordenação, padrões e integração

6.1 Tópico 8 — Padrões GoF: Observer, Mediator e Singleton

6.1.1 Como ler um padrão

Um padrão GoF é uma solução nomeada para um problema recorrente em um contexto. O catálogo clássico reúne 23 padrões em três grupos: criacionais, estruturais e comportamentais [6]. Um padrão não é

biblioteca pronta, framework nem receita obrigatória. Ele comunica intenção, participantes, colaborações, consequências e variações.

Neste recorte, **Singleton** é criacional: controla a criação de uma instância em um escopo. **Observer** é comportamental: estabelece uma dependência um-para-muitos com inscrição e notificação. **Mediator** é comportamental: centraliza a coordenação entre componentes que poderiam manter muitas referências cruzadas.

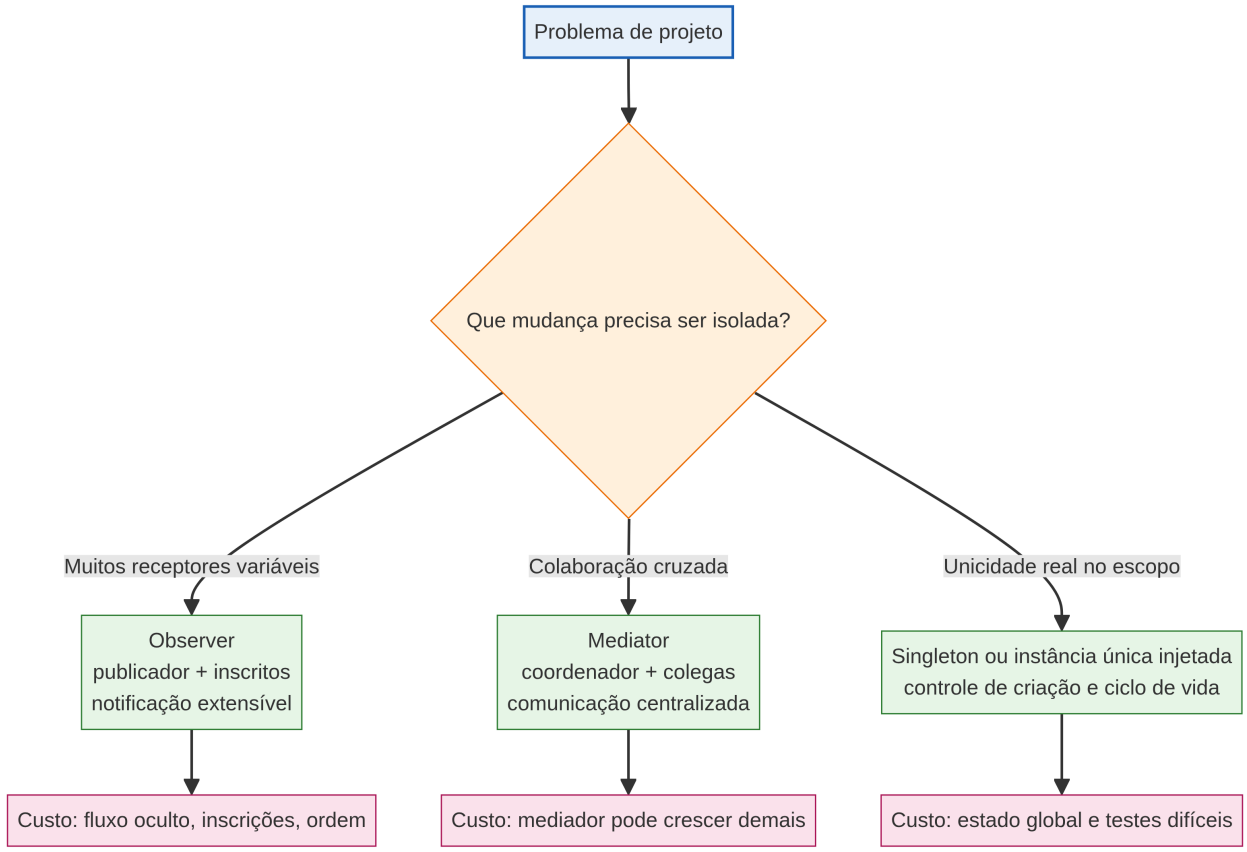


Figura 5: Árvore de decisão: o padrão deve responder a um problema descrito sem usar o nome do padrão.

6.1.2 Observer — notificação extensível

Observer define uma dependência de um-para-muitos. Quando um publicador muda ou registra um evento, os observadores inscritos podem reagir. O publicador conhece o contrato, não as classes concretas. O contrato deve dizer como inscrever, cancelar, transportar o evento, tratar exceções e encerrar o ciclo de vida.

```
public record PriceChanged(String sku, long oldCents, long newCents) {}

public interface PriceObserver {
    void onPriceChanged(PriceChanged event);
}

public final class ProductCatalog {
    private final java.util.Map<String, Long> prices =
        new java.util.HashMap<>();
```

```
    private final java.util.List<PriceObserver> observers =
        new java.util.ArrayList<>();

    public void subscribe(PriceObserver observer) {
        observers.add(java.util.Objects.requireNonNull(observer));
    }

    public void unsubscribe(PriceObserver observer) {
        observers.remove(observer);
    }

    public void changePrice(String sku, long newCents) {
        long old = prices.getOrDefault(sku, 0L);
        prices.put(sku, newCents);
        PriceChanged event = new PriceChanged(sku, old, newCents);
        for (PriceObserver observer : java.util.List.copyOf(observers))
            observer.onPriceChanged(event);
    }
}
```

A cópia da lista evita que uma inscrição ou remoção durante a notificação altere a iteração corrente. Ainda é necessário decidir reentrância, concorrência, falhas e observadores que deixam de existir. `java.util.Observable` foi depreciada desde Java 9 e não deve ser tomada como modelo atual [21]. Observer não garante por si só ordenação, entrega exatamente uma vez, persistência ou transação.

6.1.3 Mediator — coordenação centralizada

Mediator encapsula como um conjunto de objetos interage. Um componente sabe que algo aconteceu e notifica o mediador; não conhece conjuntos os colegas que serão acionados. Em uma tela, `EmailField`, `PasswordField` e `SubmitButton` podem colaborar por meio de `RegistrationMediator`.

```
interface FormMediator {
    void notify(Component sender, String event);
}

abstract class Component {
    protected final FormMediator mediator;
    protected Component(FormMediator mediator) { this.mediator = mediator; }
}

final class SubmitButton extends Component {
    SubmitButton(FormMediator mediator) { super(mediator); }
    void click() { mediator.notify(this, "click"); }
}
```

```
final class RegistrationDialog implements FormMediator {
    private final SubmitButton submit = new SubmitButton(this);

    @Override
    public void notify(Component sender, String event) {
        if (sender == submit && event.equals("click"))
            System.out.println("validar e registrar");
    }
}
```

O benefício é localizar a coordenação. O risco é criar um **God Object**. Divida mediadores por contexto, use eventos tipados em vez de strings livres quando o sistema crescer e delegue regras de domínio a serviços ou objetos especializados. Mediator não é simplesmente uma fachada: a intenção é centralizar a colaboração entre colegas [22].

6.1.4 Singleton — unicidade em escopo real

Singleton controla a criação de uma única instância em um escopo definido. "Uma instância" não significa uma instância universal para processos distribuídos, JVMs ou carregadores de classe diferentes. O padrão só é justificável quando criar instâncias adicionais causar dano real ou violar uma regra do recurso.

```
public final class AppConfig {
    private AppConfig() {}

    private static class Holder {
        private static final AppConfig INSTANCE = new AppConfig();
    }

    public static AppConfig getInstance() {
        return Holder.INSTANCE;
    }
}
```

A inicialização de classes em Java é sincronizada pela especificação; o padrão Holder aproveita essa garantia e adia a criação até a primeira chamada [23]. Um `enum` pode ser adequado para um Singleton sem estado complexo. Nenhuma dessas técnicas resolve automaticamente reflexão, múltiplos class loaders, ciclo de vida externo ou unicidade distribuída.

O principal custo é o acesso global: dependências ficam ocultas, o estado pode persistir entre testes e a substituição por um dublê fica difícil. Uma alternativa frequentemente melhor é criar uma única instância na composição da aplicação e injetá-la:

```
interface Logger { void info(String message); }

final class Service {
```

```
    private final Logger logger;
    Service(Logger logger) { this.logger = logger; }
    void execute() { logger.info("executando"); }
}

Logger logger = new ConsoleLogger();
Service serviceA = new Service(logger);
Service serviceB = new Service(logger);
```

Há uma única instância no contexto, mas `Service` não precisa conhecer uma política global de criação. Essa alternativa preserva testes isolados e dependências explícitas.

6.1.5 Comparação dos três padrões

Critério	Observer	Mediator	Singleton
Categoria	Comportamental.	Comportamental.	Criacional.
Problema	Muitos receptores variáveis.	Colaboração cruzada espalhada.	Unicidade em escopo real.
Benefício	Publicador não conhece classes concretas.	Colegas não mantêm teia de referências.	Controle de identidade e ciclo de vida.
Custo	Fluxo oculto, inscrições, ordem e falhas.	Mediador pode crescer demais.	Estado global, testes difíceis e dependências ocultas.

6.2 Como escolher um padrão sem decorar nomes

Antes de implementar, descreva o problema em uma frase sem usar o nome do padrão. Em seguida, use a tabela de decisão:

Pergunta	Candidato	Verificação seguinte
Muitos objetos precisam reagir a mudanças e o conjunto muda em tempo de execução?	Observer	Contrato do evento, cancelamento, ordem e falhas.
Componentes conhecem muitos colegas e a colaboração está espalhada?	Mediator	Coesão do mediador e risco de **God Object**.
Deve existir uma única instância em um escopo explicitamente definido?	Singleton ou instância injetada	Razão verificável para proibir instâncias adicionais.
O problema pode ser resolvido com chamada direta ou dependência injetada?	Solução simples	O padrão acrescentaria benefício mensurável?

7 Exemplos integradores

7.1 Integração 1 — Carrinho, composição e Strategy

Um carrinho contém linhas de compra. Cada linha referencia um produto independente. A regra de desconto varia e o total deve ser calculado sem conhecer todas as classes concretas de desconto.

```
public record Produto(String sku, long precoCentavos) {}

public interface Desconto {
    long aplicar(long subtotalCentavos);
}

public final class DescontoPercentual implements Desconto {
    private final int percentual;
    public DescontoPercentual(int percentual) {
        if (percentual < 0 || percentual > 100)
            throw new IllegalArgumentException("percentual inválido");
        this.percentual = percentual;
    }
    public long aplicar(long subtotal) {
        return subtotal - (subtotal * percentual / 100);
    }
}

public final class Item {
    private final Produto produto;
    private final int quantidade;
    public Item(Produto produto, int quantidade) {
        this.produto = java.util.Objects.requireNonNull(produto);
        if (quantidade <= 0) throw new IllegalArgumentException("quantidade
inválida");
        this.quantidade = quantidade;
    }
    public long subtotal() { return produto.precoCentavos() * quantidade; }
}
```

O exemplo combina classe, objeto, encapsulamento, interface, polimorfismo e composição. `Carrinho` pode compor `Item` se as linhas tiverem identidade dependente daquele carrinho; `Produto` permanece associado porque existe de modo independente. A interface `Desconto` permite variar a regra sem modificar o carrinho.

7.2 Integração 2 — Pedido, UML e invariantes

Considere as regras: cada pedido tem exatamente um cliente; um cliente pode ter zero ou muitos pedidos; cada pedido contém um ou muitos itens; cada item referencia exatamente um produto. Um caso mínimo válido é um pedido com um cliente e um item. Um pedido sem itens viola `1..*`; um item compartilhado por dois pedidos viola a composição quando a regra de domínio exigir parte exclusiva.

Instância	Estado	Válida?	Motivo

`p1`	Ligado a `c1`, com `i1`.	Sim	Caso mínimo: um cliente e um item.
`p2`	Ligado a `c1`, com `i1`, `i2`, `i3`.	Sim	Cliente pode ter vários pedidos.
`c2`	Sem pedidos.	Sim	`0..*` permite coleção vazia.
`p3`	Sem cliente.	Não	O extremo do cliente exige `1`.
`p4`	Sem itens.	Não	A composição exige `1..*`.
`i5`	Compartilhado por `p5` e `p6`.	Não	Parte composita não possui dois contêineres.

A multiplicidade deve virar uma responsabilidade de objetos. Uma coleção não deve ser exposta mutavelmente; operações como `adicionarItem` e `removerItem` precisam preservar limites, unicidade e consistência dos dois lados quando necessário.

7.3 Integração 3 — Checkout com Mediator e Observer

Um checkout pode usar Mediator para coordenar campos e botões da tela e Observer para informar a mudança de preço ao cache, à interface e à auditoria. As intenções são diferentes: o mediador coordena os componentes conhecidos do fluxo; o publicador notifica receptores que podem variar.

Uma arquitetura de referência é:

- `CheckoutDialog` implementa o contrato do mediador.
- `EmailField` e `SubmitButton` conhecem o mediador, mas não conhecem um ao outro.
- `ProductCatalog` conhece apenas `PriceObserver`.
- `CacheInvalidator` e `AuditListener` assinam eventos de preço.
- A configuração é criada na composição da aplicação e injetada, em vez de ser global por conveniência.

A revisão deve perguntar: quem cancela inscrições? O evento ocorre antes ou depois da persistência? O mediador contém coordenação ou regra de domínio? Qual é o escopo da configuração? Essas perguntas transformam um desenho de padrões em uma decisão de engenharia.

7.4 Integração 4 — Biblioteca e classe de associação

Modele `Leitor`, `Livro`, `Exemplar` e `Emprestimo` com as regras: um empréstimo é feito por exatamente um leitor; um leitor pode ter zero ou muitos empréstimos; cada empréstimo envolve exatamente um exemplar; um exemplar pertence a exatamente um livro; um livro pode ter um ou muitos exemplares; um exemplar não pode estar em dois empréstimos ativos ao mesmo tempo.

A última regra não cabe apenas em multiplicidades simples porque depende do estado ativo e da data de devolução. Ela deve ser expressa por uma restrição adicional, por contrato, consulta, teste ou regra de persistência. `Emprestimo` pode ser uma classe explícita porque o relacionamento possui datas e estado próprios.

8 Roteiro de atividades e avaliação

8.1 Atividade 1 — Diagnóstico de conceitos

Para cada afirmação, indique se é correta, incorreta ou incompleta e justifique:

1. “Classe e objeto são a mesma coisa em níveis diferentes.”
2. “Uma interface é sempre apenas uma lista de operações abstratos.”

3. “Se duas classes compartilham código, uma deve herdar da outra.”
4. “`1` no extremo Cliente significa que há um único cliente no sistema.”
5. “Qualquer coleção de filhos é composição.”
6. “Singleton é apenas uma forma elegante de escrever uma variável global.”

Uma resposta forte não se limita a marcar verdadeiro ou falso. Ela apresenta o contrato, o escopo e o caso-limite que sustentam a decisão.

8.2 Atividade 2 — Refatoração por composição

Comece com uma classe `RelatorioService` que instancia diretamente `ExportadorCsv`, `LoggerGlobal` e `RepositorioBanco`. Refatore-a para depender de interfaces e receber as implementações pelo construtor. Compare:

- quantidade de dependências concretas;
- facilidade de fornecer dublês em testes;
- clareza das pré-condições;
- possibilidade de substituir uma implementação;
- custo adicional de interfaces e composição.

8.3 Atividade 3 — Validação de diagrama UML

Modele uma plataforma de cursos. O modelo deve conter `Aluno`, `Curso`, `Disciplina` e `Matricula`. Inclua multiplicidades, papéis, ordem e unicidade quando forem relevantes. Crie quatro instâncias válidas e quatro inválidas. Explique quais regras precisam de OCL, restrição de banco ou código além da multiplicidade.

8.4 Rubrica de avaliação

Critério	Inicial	Em desenvolvimento	Adequado	Avançado
Conceitos	Confunde termos.	Define parcialmente.	Distingue conceitos.	Relaciona conceitos a contratos e limites.
Modelagem	Classes sem responsabilidades.	Relações incompletas.	Modelo coerente.	Modelo justificado por domínio e casos-limite.
Implementação	Estado exposto ou regras ausentes.	Funciona em caso comum.	Encapsula e valida.	Mantém baixo acoplamento e testabilidade.
Análise	Escolhas sem justificativa.	Reconhece alguns riscos.	Explica trade-offs.	Compara alternativas e prevê manutenção.
Comunicação	Diagrama ou código ilegível.	Documentação irregular.	Apresentação clara.	Argumentação precisa, rastreável e referenciada.

9 Síntese final e checklist

Orientação a Objetos não é uma coleção de palavras-chave. É uma disciplina de modelagem. Classes e objetos estabelecem o vocabulário; operações e contratos definem ações; encapsulamento protege invariantes; interfaces e classes abstratas estabilizam pontos de variação; herança só é legítima quando a substituibilidade comportamental se mantém; polimorfismo permite ao cliente depender da abstração; UML torna relações e limites visíveis; associação, agregação e composição devem expressar semânticas diferentes; padrões GoF reorganizam acoplamento, coordenação ou criação, cada um com custos permanentes.

Antes de entregar um projeto, verifique:

- cada classe tem responsabilidade compreensível e coesa;
- cada operação pública tem pré-condições e pós-condições identificáveis;
- invariantes não podem ser quebradas por setters ou referências expostas;
- interfaces descrevem contratos reais, não abstrações artificiais;
- herança é sustentada por uma relação de subtipagem, não somente por reutilização;
- polimorfismo é usado para variar comportamento, não para esconder casts em série;
- multiplicidades foram lidas a partir da instância correta;
- cardinalidade foi separada de ordem, unicidade e navegabilidade;
- composição e agregação têm justificativa de domínio;
- relações com atributos próprios foram modeladas como classes de associação;
- Observer tem cancelamento e contrato de entrega;
- Mediator não virou um objeto central sem coesão;
- Singleton tem escopo e requisito de unicidade explícitos;
- alternativas mais simples foram consideradas;
- diagramas, código, testes e referências contam a mesma história.

10 Referências

As referências abaixo indicam o referencial teórico e normativo utilizado. As fontes de linguagem e UML sustentam afirmações específicas; os textos acadêmicos sustentam as distinções conceituais; o livro GoF sustenta o vocabulário e as intenções dos padrões.

1. Luca Cardelli e Peter Wegner. **On Understanding Types, Data Abstraction, and Polymorphism**. ACM Digital Library.
2. David L. Parnas. **On the Criteria to Be Used in Decomposing Systems into Modules**. Communications of the ACM.
3. Alan Snyder. **Encapsulation and Inheritance in Object-Oriented Programming Languages**. OOPSLA.
4. Barbara H. Liskov e Jeannette M. Wing. **A Behavioral Notion of Subtyping**. Carnegie Mellon University.
5. Object Management Group. **Unified Modeling Language, Version 2.5.1**. Especificação formal.
6. Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides. **Design Patterns: Elements of Reusable Object-Oriented Software**. The Hillside Group.
7. Oracle. **The Java Language Specification, Java SE 21, Chapter 8: Classes**. JLS Chapter 8.

8. Oracle. **The Java Language Specification, Java SE 21, Chapter 9: Interfaces**. JLS Chapter 9.
9. Oracle. **The Java Language Specification, Java SE 21, Chapter 15: Expressions**. JLS Chapter 15.
10. Oracle. **Interfaces — The Java Tutorials**. Oracle Java Tutorials.
11. Oracle. **Class AbstractList — Java SE API Specification**. Java API.
12. Oracle. **Multiple Inheritance of State, Implementation, and Type**. Java Tutorials.
13. Python Software Foundation. **The Python 2.3 Method Resolution Order**. Python documentation.
14. ISO. **ISO/IEC 19505-1:2012 — Information technology — OMG UML**. International Organization for Standardization.
15. Oracle. **Type Erasure — The Java Tutorials**. Generics.
16. Oracle. **Effects of Type Erasure and Bridge Methods**. Generics.
17. Donald Bell. **The UML 2 Class Diagram**. IBM Developer.
18. Oracle. **Interface Set — Java Platform API**. Java API.
19. Oracle. **Interface List — Java Platform API**. Java API.
20. Jordi Cabot, Robert Clariso e Daniel Riera. **On the Verification of UML/OCL Class Diagrams using Constraint Programming**. Journal of Systems and Software.
21. Oracle. **Class Observable — Java SE 17 API**. Java API.
22. Refactoring.Guru. **Mediator**. Referência técnica de padrões.
23. Oracle. **Chapter 12: Execution — Java Language Specification**. Inicialização de classes.
24. Refactoring.Guru. **Observer**. Referência técnica de padrões.
25. Martin Fowler. **Collections On Class Diagrams**. Bliki de modelagem.
26. Oracle. **Object-Oriented Programming Concepts**. Java Tutorials.

10.1 Nota sobre o alinhamento

A planilha enviada é a fonte de alinhamento curricular deste documento. A matriz é tratada como especificação de escopo: oito tópicos, quatro unidades, quatro objetivos de aprendizagem e uma distribuição cognitiva sugerida. O referencial teórico externo foi usado para ampliar, explicar e validar os conceitos; ele não altera os tópicos definidos na matriz.